MEMORIA TÉCNICA OSO

INTRODUCCIÓN:

La estructura del proyecto se divide en dos carpetas, oso.net donde se guardan las conexiones y los ejecutables de los servidores y oso.ui donde se guardan los ejecutables y clases útiles para estos, que usarán los clientes.
Antes de empezar, explicaré brevemente cómo está estructurado el proyecto. Comenzaré por las clases útiles para todo el proyecto que son: tablero, clienteOso, cliente y jbuttonOso. Luego, explicaré las clases principales como OsoGUI, sTcpOsoServerMultiThread, TcpOsoConexionMultiThread, TcpChatServerMultiThread, TcpChatConexionMultiThread.
El resto de clases como OsoSimple, Chat, OsoSolitarioGui,etc. No las explicaré, ya que son clases que su lógica está implementada en OsoGUI, puesto que mi objetivo fue intentar incluir toda la lógica en esa clase y que puedas acceder a distintos modos de juegos.

TABLERO:

Empezaré explicando la elección de las funciones y atributos de la clase de Tablero, que es la clase encargada de guardar, gestionar y actualizar la lógica de la partida. Para esto, necesitaremos la dimensión del tablero y el propio tablero, que se representará con una matriz de caracteres. Estos dos atributos serían los iniciales, luego usaría algunos atributos auxiliares como tableroAux, que es un tablero que guardará el estado de las posiciones siendo 0 libre y 1 con un oso; y el entero oso en el cual se guardará la cantidad de puntos.

```
char tablero[][];
int tableroAux[][]; //0 libre, 1 aliados, 2 enemigos, 3 en uso
int x,y,osos,ososA,ososE;
String nombre;
```

Las variables ososA, osoE y nombre son variables para el multijugador y los estado 1,2,3 representan distintos estados de las posiciones siendo 0 libre, 1 oso aliado, 2 oso enemigo y 3 con letra pero no forma oso.
Inicialmente la clase estaba pensada para una persona, sin embargo esto luego será modificado para más de un jugador, es por ello que la clase tabla tiene dos constructores:

```
public Tablero(int x, int y){...14 lines }

public Tablero(int x, int y, String nombre){...16 lines }
```

El primer constructor es para jugar un jugador, por lo tanto, solo se necesita las dimensiones del tablero. El segundo constructor tiene una variable extra, que es el nombre. Este nombre hace referencia al dueño del tablero, es decir el jugador que juega con este tablero, ya que al ser un juego online pensé en seguir la lógica de algunos juegos multijugador en el que hay dos equipos, azul y rojo, pero desde la vista del usuario tu siempre eres el azul y el enemigo el rojo.
La logia del tablero es sencilla, tiene estas funciones principales:

```
public boolean ponerPieza(int x, int y, char pieza){...11 lines }

public boolean tableroLleno(){...9 lines }

public void comprobarOso(){...52 lines }

public void comprobarOso(String nombre){...53 lines }
```

estas, hacen lo que pone en el nombre, poner pieza pone la pieza ‘pieza’ en la posición x,y; tableroLleno te devuelve si el tablero está lleno; luego, las funciones comprobarOso actualiza las variables oso, osoA, osoE con el número de osos aliado o enemigos. Si no le pasas nombre, actualizará la variable oso sumándole uno cuando encuentre un oso; Si le pasa un nombre, sumará 1 a osoA si el nombre es igual al nombre del tablero, sino, sumará uno a osoE. Los osos se calculan buscando s en el tablero, y si esta forma la palabra OSO con las letras de los lados, sumará un oso que cambiará los números de la tablaAUX y saldrá de la función, aquí un ejemplo de cálculo de oso para una diagonal:

```
for(int i=0;i<this.x;i++){
    for(int j=0;j<this.y;j++){
        if(this.tablero[i][j] == 'S'){
            if(i!=0 && i!=this.x-1 && j!=0 && j!=this.y-1){
                oso = "" + this.tablero[i-1][j-1]+this.tablero[i][j]+this.tablero[i+1][j+1];
                libre = (this.tableroAux[i-1][j-1] != 1 && this.tableroAux[i][j] != 1 && this.tableroAux[i+1][j+1] != 1);
                if(oso.equals("OSO") && libre){
                    this.tableroAux[i-1][j-1] = 1;
                    this.tableroAux[i][j] = 1;
                    this.tableroAux[i+1][j+1] = 1;
                    this.osos++;
                    return;
                }
```

La variable aux es para el multijugador y se calcula con una función lambda, en solitario aux se cambia por 1.

```
int aux = (nombre.equals(this.nombre)) ? 1: 2;
```

El tablero representado quedaría de esta manera:

```
-----TABLERO-----
O,S,O, , , ,
 , , , , , ,
 , , , , , ,
 , ,S, , , ,
 , ,O, , , ,
-----------------
-----TABLEROAUX-----
1,1,1,0,0,0,
0,0,0,0,0,0,
0,0,0,0,0,0,
0,0,3,0,0,0,
0,0,3,0,0,0,
-----------------
```

Cliente, TcpChatServerMultiThread y TcpChatConexionMultiThread:

Estas tres clases se encargan de gestionar el servidor y los clientes del chat del juego.
La clase TcpChatServerMultiThread se encarga de aceptar los clientes nuevos que quieran conectarse al chat; La clase TcpChatConexionMultiThread se encarga de recibir y reenviar los mensajes de los clientes al resto de clientes, y, por último, la clase cliente se encarga de enviar los mensajes del usuario a conexión y recibir e imprimir los mensajes recibidos de conexión. La clase Cliente y TcpChatConexionMultiThread son hilos de los procesos principales, cliente de OsoGUI y TcpChatConexionMultiThread de TcpChatServerMultiThread.

ClienteOso, ClienteOsoBot, TcpOsoServerMultiThread y TcpOsoConexionMultiThread:

Estas clases al igual que las anteriores se encargan de gestionar el servidor y los clientes de la partida de oso. Empezaré explicando la lógica del servidor.

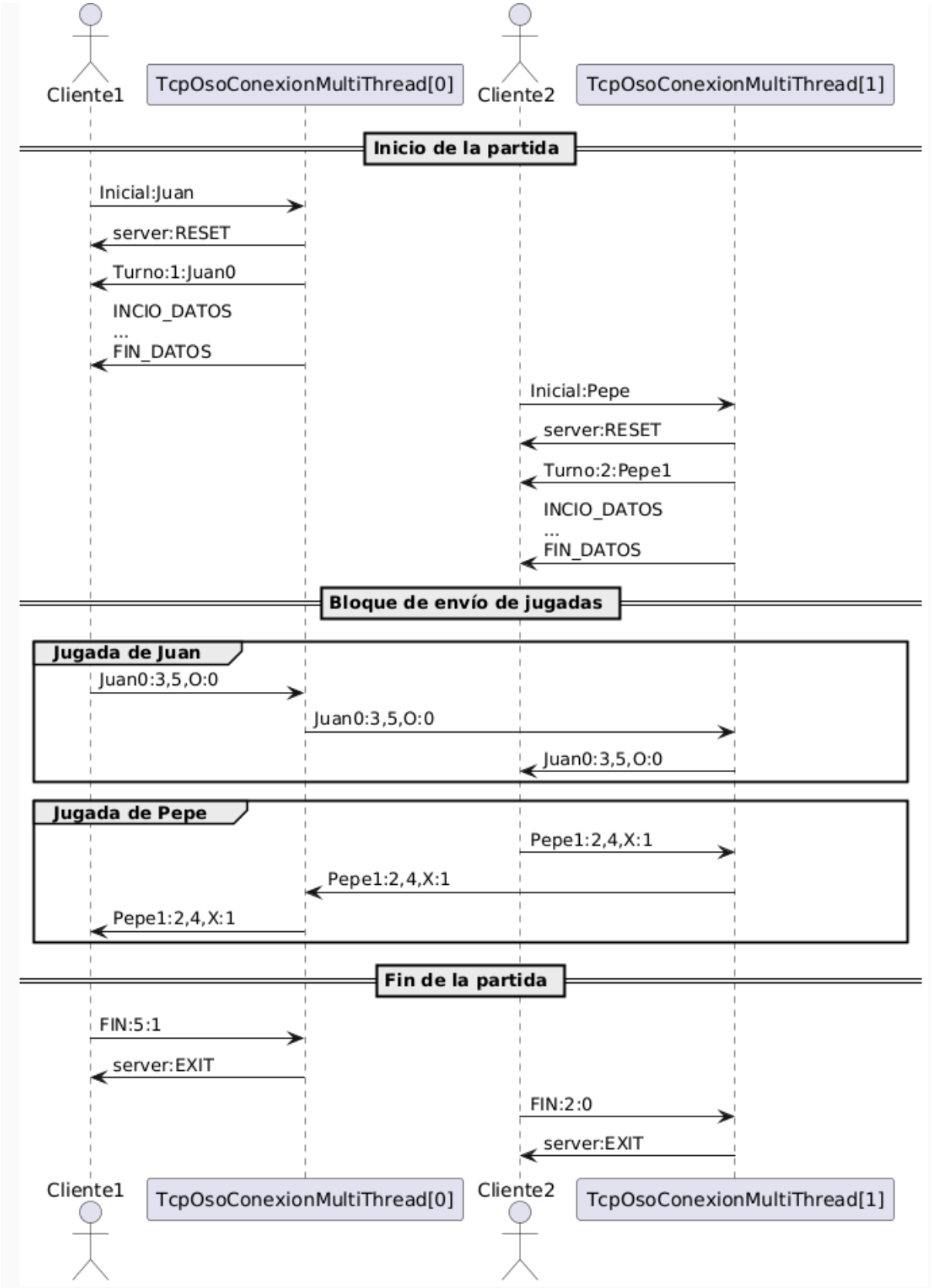
El servidor se encarga igual que en el chat de aceptar los clientes, con dos pequeñas diferencias. La primera, es que a diferencia del chat, este servidor no crea un hilo de conexión cada vez que se conecta un usuario, sino cuando al menos haya dos. Esta es gestionado con una lista, que al igual que un videojuego, el cliente al buscar partida entra en una lista la cual almacenará su socket y en el momento que se conecte otro jugador al socket, empezará la partida creando dos hilos de conexión. Estos se podrán comunicar con sus contrincantes mediante una tabla de tamaño dos de TcpOsoConexionMultiThread.

```
List<Socket> clientesPendientes;
```

```
TcpOsoConexionMultiThread[] partida = new TcpOsoConexionMultiThread[2];
```

La otra diferencia, es que antes de abrir el servidor, carga en una variable HashMap un serializable Ganadores.ser, donde se guardan los ganadores siendo el nombre la key y las partidas ganadas sus value, esto por la sencillez de usar variables serializadas y que el archivo no tiene que ser legible desde el servidor si no desde el cliente.

El funcionamiento de ClienteOso y TcpOsoConexionMultiThread va muy relacionado, un diagrama UML de ejemplo del funcionamiento de estas clases puede ser:



En el bloque “inicio de partida”, los clientes le mandan un mensaje inicial al servidor con su nombre, el servidor les responde con un RESET para que receten el tablero y un mensaje

TURNO: nombre+index:turno, para que los clientes sepan su turno y su nombre más el índice para que no haya conflictos en la función comprobarOso.
En el bloque de envío de jugadas los jugadores enviaron mensajes con NombreDePartida:Movimiento:SiRepiteONo.
En el bloque final, los jugadores cuando noten que su tablero está lleno, se enviará al servidor un mensaje con FIN:NumeroDeOso:SiGananONo(en el ejemplo de arriba gana Juan con 5 osos y pierde Pepe con 2). Luego, el server les contestará con un EXIT que hará que el cliente se reencole en la lista de clientes pendientes.

Cuando el servidor recibe FIN re escribirá el mapa y lo guardará en el archivo Ganadore.ser; si el servidor nota (mediante un try catch) que falta un jugador le mandará un mensaje EXIT al cliente y el registroOso actualizado.
Por su contraparte, el cliente guardará los ganadores en RegistroOso.txt, aparte de la gestión del registro y enviar mensajes, el cliente también se encarga de actualizar el tablero cada vez que recibe un comando. Es capaz de esto ya que en el constructor le pasamos el JPanel y los JTextField que contiene el registro de puntos enemigos y aliados.

```
public ClienteOso(String nombre, String add, int port, Tablero tablero, JPanel jPanelTablero, JTextField jTextFieldOsoA, JTextField jTextFieldOsoE)
```

El clienteOsoBot funciona igual que un clienteOso, con la peculiaridad de que no necesita un GUI y manda los mensajes solos, además de que no guarda el fichero RegistroOso.txt.

```
/*MinimaxResult result = this.iA.calcularMovimientoMinMax(tablero, -999, 999,this.nombre+"IA", 5);
int[] mov = result.mejorMovimiento;*/
```

Este no es herencia de clienteOso, ya que al no necesitar casi ningún atributo de este, pensé que iba a ser más rápido creando una clase nueva.

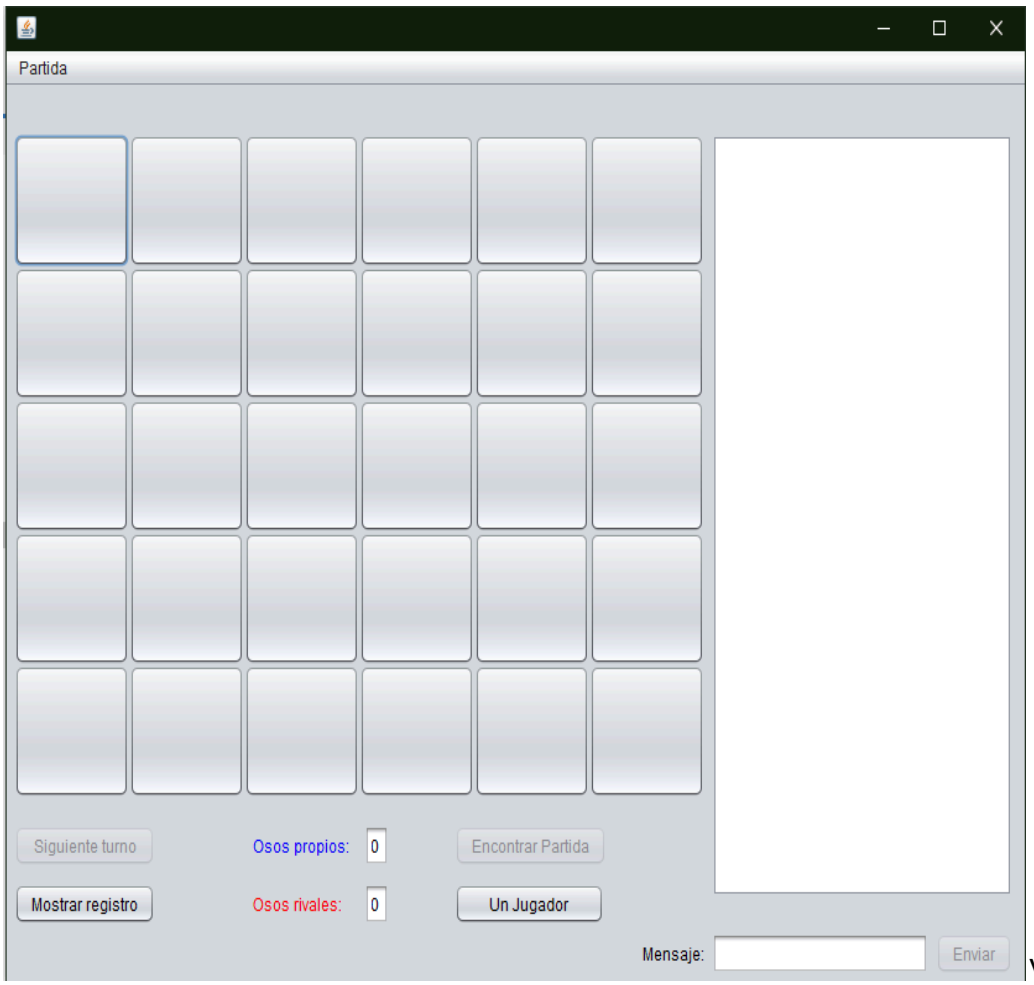
La lógica es así, ya que en la asignatura de redes hemos hecho un chat que funciona bastante parecido. Este puede enviar ficheros teniendo la misma lógica que uso en este código así como la gestión de acciones de los clientes mediante los comandos.

IAOso:

Inicialmente pensaba que el bot tendría que tener algún tipo de inteligencia, así que usé un proyecto de IA en el cual implementamos un algoritmo de búsqueda MiniMax. En el código, esta implementación está comentada y el bot funciona aleatoriamente como pone en el enunciado de la práctica. El cálculo de movimientos es muy sencillo, crea una tabla con las posiciones que están representadas con un 0 en el tablero Aux y luego, mediante un random, elige un índice. Para no complicarme mucho, le puse que devuelva una tabla de 3 enteros que los dos primeros representan las coordenadas y el tercero la letra, siendo 0 “O” y 1 “S”

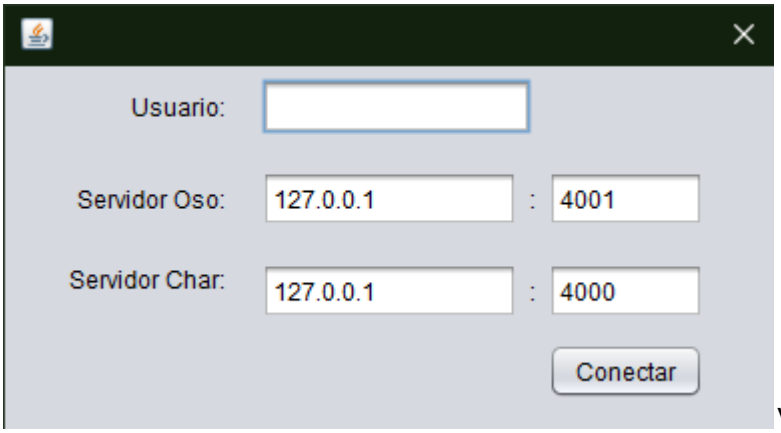
OsoGUI, JButtonOso:

Para poder explicar el GUI principal, comenzaré por cómo funciona y que ofrece el GUI.



Ventana inicial.

Menú de la partida.



Ventana de configuración

Inicialmente, cuando abres el GUI, te mostrará un tablero el cual podrás jugar en modo solitario hasta que te conecte y busques partida o hasta que le des a un jugador. Si estás conectado, la opción “buscar partida” se activará y podrás entrar en cola para jugar. Si le das a “Un jugador”, jugarás en local contra un BOT. A la derecha se puede ver el menú del

chat para mandar y recibir mensajes o donde también se mostrará el registro de victorias si le das a Mostrar Registro, te mostrará el registro que tenga el cliente en memoria.
Toda la implementación del GUI ha sido realizada mediante el actionPerformed(), ya que en la práctica 5 implementamos el GUI con ActionListener y como no sabia de otra manera, empecé así y acabé el proyecto con él.
Si tienes un nombre no nulo y las ip y puertos hacen referencia a un servidor abierto, te dejará conectar, entonces, el programa intenta desconectarse del servidor del chat. Si no puede, no pasara nada, luego instancia el cliente y lo inicia. Una vez conectado podrás darle a buscar partida, entonces el programa intentará desconectarse del servidor del oso, si no puede, no pasara nada. Luego, instancia el clienteOso y lo inicia, esto simula buscar partida y entrar en cola de los videojuegos online.
El botón envía un mensaje al contrincante RESET y resetea el tablero de la partida y desconectar cierra los Clientes que envían un mensaje de FIN al contrincante.
Finalmente, los botones del panel gestionan los turnos de juego, ya que sin estos, los jugadores podrían pasar comando sin ningún tipo de orden. Los botones son de la clase JButtonOso, que funciona como un botón pero con ciertas modificaciones.

```
public JButtonOso(int x, int y, Tablero tablero)
```

Este es su constructor, el tablero es el tablero de la partida y los enteros x, y es la posición que representan. Tiene tres funciones importantes, la primera darValor, que alterna el valor del botón entre “”, “O” y “S”. Esto nos permite alternar el texto del botón clicando en el. La función actualizarTablero pone la pieza que representa el botón en el tablero del juego y finalmente la función actualizarBoton, que actualiza el botón con el valor correspondiente del tablero. Estas funciones se usan tanto para gestionar los turnos como para actualizar el tablero de la siguiente manera: Al cliente que le toque el turno 1, empezará con todos lo botones del tablero activados, estos, se desactivarán y una vez elegida la jugada y darle a siguiente turno, desactivando así los botones de su panel, el contrincante recibirá el mensaje y el programa actualizara el tablero y el panel y activará sus botones. Así sucesivamente hasta que se acabe la partida.

Conclusión:

El diseño del GUI está diseñado como a mí me parece cómodo. La elección de ir cambiando el texto mediante clics me resulta más cómoda que elegir la letra de antes, el encolar. La posibilidad de jugar contra un bot en local y la capacidad de jugar en solitario si no quiero contrincante, es porque si yo fuera cliente me parecería cómodo. La lógica interna del código está basada en mis conocimientos aprendidos en clase, tanto la lógica del GUI con el actionListener y la lógica de envío de comando entre los clientes, son cosas que he usado en la prácticas de esta u otra asignatura.